

Sistem de Gestionare a unei Biblioteci

Documentație Proiect POO — C++17

Student: Ursescu Timotei

Grupă: 3122A

Universitate: Universitatea Ștefan cel Mare, Suceava

Limbaaj: C++17

Mediu: WSL Ubuntu / VS Code

Repository: branch develop — GitHub

1. Cerința Proiectului

Implementarea unui sistem pentru o bibliotecă virtuală cu clase pentru cărți, utilizatori și împrumuturi, demonstrând conceptele fundamentale ale programării orientate pe obiecte în C++17.

Cerințe obligatorii acoperite:

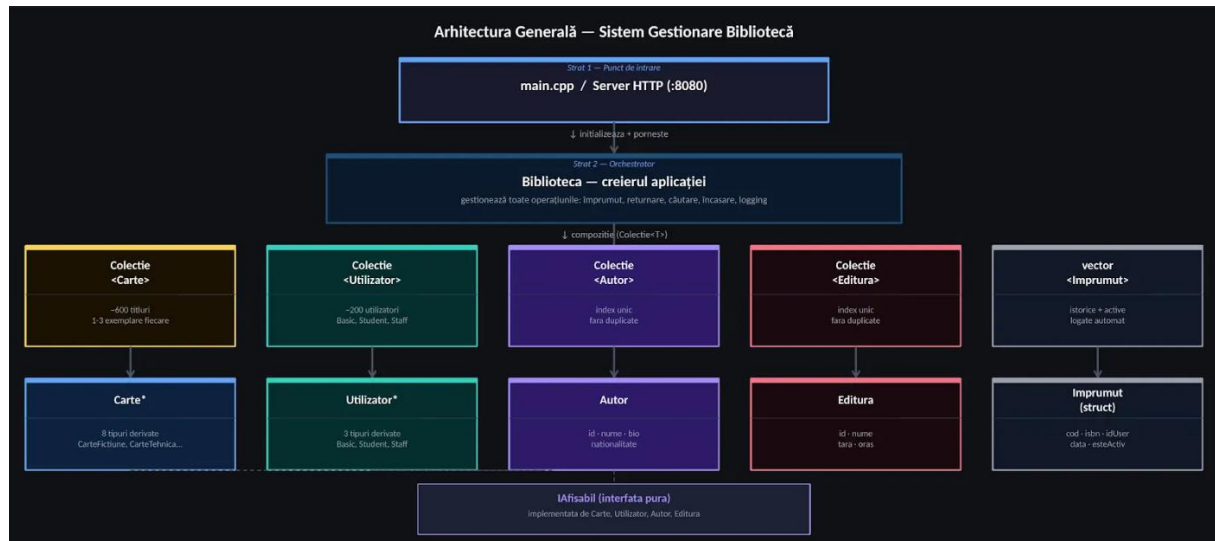
Cerință	Fișier / Implementare
Clase: Carte, Utilizator, Biblioteca	<code>Carte.h</code> , <code>Utilizator.h</code> , <code>Biblioteca.h</code>
Moștenire pentru tipuri de cărți	<code>CarteFictiune.h</code> , <code>CarteTehnica.h</code> , etc.
Polimorfism — metode virtuale	<code>afisareDetalii()</code> , <code>getTipCarte()</code> , etc.
Encapsulare — attribute private + get/set	Toate clasele
Evenimente logate	<code>Biblioteca.h</code> → <code>biblioteca_log.txt</code>
Teste unitare	<code>TestStoc.cpp</code> , <code>TestImprumuturi.cpp</code> , <code>TestPolimorfism.cpp</code>
Git — 5+ commit-uri, branch develop	Repository GitHub

Cerințe facultative acoperite:

Cerință	Implementare
Șablon generic pentru stocare	<code>Colectie<T></code> în <code>Colectie.h</code>
Excepții personalizate	5 clase în <code>Exceptii.h</code>

2. Arhitectura Generală

Aplicația este structurată în trei straturi principale: punctul de intrare (`main.cpp`), orchestratorul central (`Biblioteca`) și entitățile de domeniu. Comunicarea cu utilizatorul se realizează printr-o interfață web accesibilă la `http://localhost:8080`.



Toate clasele afișabile implementează interfața `IAfisabil`, iar stocarea omogenă a obiectelor polimorfe se face prin `Colecție<T>`.

3. Concepte POO Implementate

3.1 Encapsulare

Toate atributele claselor sunt declarate `private` sau `protected`. Accesul extern se face exclusiv prin metode `get/set` publice. Cazul cel mai relevant este parola utilizatorului:

```
private:
    string parolaHash; // parola nu e niciodata in clar

public:
    void setParola(const string& parolaNoua); // face hash automat
    bool verificaParola(const string& parola) const; // nu expune hash-ul
```

`parolaHash` este privat și nu există `getter` pentru ea. Securitatea este încapsulată în namespace-ul `Securitate` din `Utilizator.h`.

3.2 Moștenire

Există două ierarhii principale de moștenire:

Ierarhia Carte:

```
IAfisabil (interfata pura)
+-- Carte (clasa abstracta)
    |-- CarteFictiune      - gen literar, varsta minima
    |-- CarteTehnica       - domeniu, dificultate, resurse digitale
    |-- CarteEducativa     - materie, profil, clasa
    |-- CarteCopii         - varsta recomandata, ilustrator
    |-- MaterialeReferinta - doar sala de lectura, 0 zile
    |-- CarteReligioasa    - religie, cult
    |-- Periodic           - ISSN in loc de ISBN, 0 zile
+-- ManuscrisRar          - acces restrictionat, 0 zile
```

Ierarhia Utilizator:

```
IAfisabil (interfata pura)
+-- Utilizator (clasa abstracta)
    |-- UtilizatorBasic    - 2 carti, fara discount
    |-- UtilizatorStudent  - 5 carti, discount 20%
+-- UtilizatorStaff       - 15 carti, fara taxe, drepturi admin
```

3.3 Polimorfism

Polimorfismul se manifestă în mai multe locuri cheie:

La afișare — același apel produce output diferit per tip:

```
vector<Carte*> carti = { new CarteFictiune(...), new CarteTehnica(...) };
for (Carte* c : carti)
    c->afisareDetalii(); // output diferit pentru fiecare tip
```

La calcul taxe — suprascrisă diferit per tip:

```
double taxaFinala = u->aplicaDiscountTaxe(taxaBruta);
// UtilizatorStudent -> taxaBruta * 0.8 (discount 20%)
// UtilizatorStaff   -> 0.0 (scutit complet)
```

Tip carte	Zile	Taxă/zi
CarteFictiune	14	1.0 RON
CarteTehnica	28	2.0 RON

Tip carte	Zile	Taxă/zi
CarteEducativa	30	1.5 RON
CarteCopii	14	0.5 RON
MaterialeReferinta	0 (sală)	5.0 RON
CarteReligioasa	21	1.0 RON
Periodic	0 (sală)	2.0 RON
ManuscrisRar	0 (restricționat)	50.0 RON

3.4 Abstractizare

Carte și Utilizator sunt clase abstracte — nu pot fi instanțiate direct. Conțin metode virtuale pure care forțează clasele derivate să implementeze comportamentul specific:

```
// In Carte.h
virtual int      getTimpImprumut()    const = 0;
virtual double   getTaxaDeteriorare() const = 0;
virtual double   getTaxaIntarziere()  const = 0;
virtual string    getTipCarte()       const = 0;
```

3.5 Interfața IAfisabil

Separată de ierarhia de moștenire, `IAfisabil` este o interfață pură — clasă fără date, doar metode virtuale pure. Atât `Carte` cât și `Utilizator`, `Autor`, `Editura` o implementează:

```
class IAfisabil {
public:
    virtual void afisareDetalii() const = 0;
    virtual ~IAfisabil() = default;
};
```

3.6 Supraîncărcarea Operatorilor

Operatorul `<<` este supraîncărcat pentru afișare rapidă la `Carte` și `Utilizator`:

```
friend std::ostream& operator<<(std::ostream& os, const Carte& c) {
    os << "[" << c.getTipCarte() << "]" << c.titlu
    << " (" << c.anAparitie << ") - ISBN: " << c.isbn;
    return os;
}
// Utilizare:
cout << *unaCarte << "\n"; // [Fictiune] Dune (2020) - ISBN: ISBN-F001
```

3.7 Șabloane (Templates)

`Colectie<T>` este o clasă template generică ce înlocuiește `vector<T*>` brut din `Biblioteca`. Avantaje: gestionare automată a memoriei în destructor, interdicție copiere (previne double-delete), validare null la adăugare, excepție la index invalid:

```
Colectie<Carte>    inventar;        // stocheaza Carte*
Colectie<Utilizator> utilizatori;    // stocheaza Utilizator*
Colectie<Autor>    indexAutori;      // stocheaza Autor*

for (auto* carte : inventar)          // range-based for
    carte->afisareDetalii();
```

4. Descrierea Claselor

4.1 Interfața IAfisabil

Fișier: `src/IAfisabil.h` — clasă abstractă pură care definește contractul de afișare.

Metodă	Tip	Descriere
<code>afisareDetalii()</code>	virtual pur	Afișează informațiile complete ale obiectului
<code>~IAfisabil()</code>	virtual	Destructor virtual — permite delete polimorfic

4.2 Șablonul Colectie<T>

Fișier: `src/Colectie.h` — container generic pentru stocare și gestionare de pointeri polimorfici.

Metodă / Atribut	Tip	Descriere
<code>elemente</code>	<code>vector<T*></code>	Vectorul intern de pointeri
<code>adauga(T*)</code>	<code>void</code>	Adaugă element; aruncă <code>invalid_argument</code> dacă e null
<code>elimina(T*)</code>	<code>bool</code>	Șterge și dealocă elementul; returnează succes
<code>get(int)</code>	<code>T*</code>	Acces prin index; aruncă <code>out_of_range</code> dacă invalid
<code>dimensiune()</code>	<code>int</code>	Numărul de elemente
<code>esteGoala()</code>	<code>bool</code>	Verificare colecție vidă
<code>begin() / end()</code>	iterator	Permite range-based for

Constructorul de copiere și operatorul = sunt șterse (= delete) pentru a preveni dubla alocare a memoriei.

4.3 Entitățile de Bază: Autor și Editura

Fișiere: `src/Autor.h`, `src/Editura.h` — entități independente cu ID auto-generat printr-un contor static. `Biblioteca` menține câte un `Colectie<Autor>` și `Colectie<Editura>` care garantează unicitatea.

Atribut	Tip	Descriere
<code>id</code>	<code>int (static)</code>	Auto-generat prin contor static
<code>nume, prenume</code>	<code>string</code>	Numele complet al autorului
<code>nationalitate</code>	<code>string</code>	Țara de origine
<code>anNastere</code>	<code>int</code>	Anul nașterii
<code>esteViu</code>	<code>bool</code>	Autor în viață sau decedat
<code>bio</code>	<code>string</code>	Scurtă biografie
<code>linkProfil</code>	<code>string</code>	Link Wikipedia/Goodreads etc.

Tip derivat	Specific	Zile	Taxă/zi
CarteTehnica	domeniu, nivelDificultate, areResurseDigitale	28	2.0 RON
CarteEducativa	materie, profil, clasa	30	1.5 RON
CarteCopii	varstaRecomandata, ilustrator, areElementeInteractive	14	0.5 RON
MaterialeReferinta	tipReferinta	0 (sală)	5.0 RON
CarteReligioasa	religie, cult	21	1.0 RON
Periodic	issn, tipPeriodic, nrEditie	0 (sală)	2.0 RON
ManuscrisRar	epoca, conditiiPastrare, necesitaSupervizare	0	50.0 RON

4.5 ExemplarFizic și Sistemul de Coduri Unice

Definit în: `src/Carte.h` (struct). ISBN-ul identifică titlul. `codUnic` identifică copia fizică individuală.

Format cod unic: `BIB-YYYY-NNNNN` (ex: `BIB-2024-00001`) — generat automat de `Biblioteca::genereazaCodExemplar()`. Nu poate exista același cod de două ori.

Atribut	Tip	Descriere
<code>codUnic</code>	<code>string</code>	Identificator unic al copiei (BIB-YYYY-NNNNN)
<code>status</code>	<code>StatusCarte</code> enum	DISPONIBILA, IMPRUMUTATA, REZERVATA, DETERIORATA, SCOASA_DIN_UZ
<code>locatieCladire</code>	<code>string</code>	Clădirea unde se află exemplarul
<code>locatieCamera</code>	<code>string</code>	Sala / camera din clădire
<code>locatieRaft</code>	<code>string</code>	Raftul specific
<code>locatieSectiune</code>	<code>string</code>	Secțiunea raftului
<code>provenienta</code>	<code>ProvenientaCarte</code> enum	CUMPARATA, DONATIE, SCHIMB, MOSTENIRE
<code>anIntrareColectie</code>	<code>int</code>	Anul intrării în colecție
<code>pretAchizitie</code>	<code>double</code>	Prețul plătit de bibliotecă (poate diferi de <code>pretCatalog</code>)
<code>observatii</code>	<code>string</code>	Notițe despre stare (ex: pagina 34 patată)

La împrumut, serverul găsește codul exemplarului și returnează locația fizică — afișate utilizatorului în interfața web.

4.6 Clasa Utilizator și Ierarhia Derivată

Fișier: `src/Utilizator.h` — clasă abstractă de bază.

Atribut	Tip	Descriere
<code>id</code>	<code>string</code>	Identificator unic (ex: STU-1001, STAFF-001)
<code>nume, prenume</code>	<code>string</code>	Numele complet
<code>contact</code>	<code>string</code>	Email sau telefon
<code>tip</code>	<code>TipUtilizator</code> enum	BASIC, STUDENT, STAFF
<code>taxeAcumulate</code>	<code>double</code>	Suma totală de plată (RON)
<code>parolaHash</code>	<code>string</code>	Parola stocată ca hash hex pe 16 caractere
<code>anInregistrare</code>	<code>int</code>	Anul înregistrării în sistem
<code>istoric</code>	<code>vector<EvenimentIstoric></code>	Istoricul evenimentelor, sortat cronologic automat
<code>imprumututiCurente</code>	<code>vector<string></code>	Codurile exemplarelor împrumutate activ

Tip utilizator	Limită cărți	Discount taxe	Admin
<code>UtilizatorBasic</code>	2 cărți	0%	Nu

Tip utilizator	Limită cărți	Discount taxe	Admin
UtilizatorStudent	5 cărți	20%	Nu
UtilizatorStaff	15 cărți	100% (scutit)	Da

4.7 Istoric și Securitate

Istoricul evenimentelor (struct EvenimentIstoric):

Fiecare utilizator are un vector de evenimente cu an, lună și descriere. La fiecare `adaugaEveniment()`, vectorul este sortat automat cronologic după an și lună — indiferent de ordinea inserției. Permite urmărirea evoluției în timp.

Securitate parole (namespace Securitate):

- Parola nu se stochează niciodată în clar
- `hashParola()` aplică `std::hash<string>` și returnează reprezentarea hex pe 16 caractere
- `verificaParola()` hash-uieste parola introdusă și compară cu hash-ul stocat
- `parolaHash` este privat și nu există getter — parola nu poate fi citită din exterior

4.8 Clasa Biblioteca

Fișier: `src/Biblioteca.h` — clasa centrală care orchestrează toate operațiunile.

Atribut	Tip	Descriere
<code>inventar</code>	<code>Colectie<Carte></code>	Toate titlurile din bibliotecă
<code>utilizatori</code>	<code>Colectie<Utilizator></code>	Utilizatori înregistrați
<code>indexAutori</code>	<code>Colectie<Autor></code>	Index unic de autori
<code>indexEdituri</code>	<code>Colectie<Editura></code>	Index unic de edituri
<code>imprumuturi</code>	<code>vector<Imprumut></code>	Istorie și active
<code>totalIncasari</code>	<code>double</code>	Suma totală colectată
<code>contorExemplare</code>	<code>int</code>	Contor pentru generare coduri unice BIB-YYYY-NNNNN
<code>logFile</code>	<code>ofstream</code>	Fișier log (biblioteca_log.txt)

Metodă	Descriere
<code>gasesteOrAdaugaAutor(...)</code>	Asigură unicitatea autorilor în index
<code>adaugaCarte(Carte*)</code>	Verifică ISBN duplicat, adaugă în inventar
<code>adaugaExemplarLaCarte(...)</code>	Generează cod unic BIB-YYYY-NNNNN, adaugă exemplar fizic
<code>imprumutaCarte(idUser, isbn, ...)</code>	Verifică limite, disponibilitate, actualizează status
<code>returneazaCarte(idUser, cod, zile)</code>	Actualizează status, calculează taxe cu discount
<code>incaseazaTaxa(idUser, suma)</code>	Scade din taxele utilizatorului, adaugă la încasări
<code>cautaDupaTitlu(fragment)</code>	Căutare parțială în titlu (case-insensitive)
<code>cautaDupaAutor(numeAutor)</code>	Căutare după autor
<code>cautaDupaAn(an)</code>	Filtrare după an apariție

Metodă	Descriere
afiseazaStatistici()	Raport general: cărți, utilizatori, încasări
Logging: fiecare operație importantă (adăugare, împrumut, returnare, încasare) se înregistrează automat în biblioteca_log.txt.	

4.9 Excepții Custom

Fișier: src/Exceptii.h — toate derivă din `std::runtime_error`.

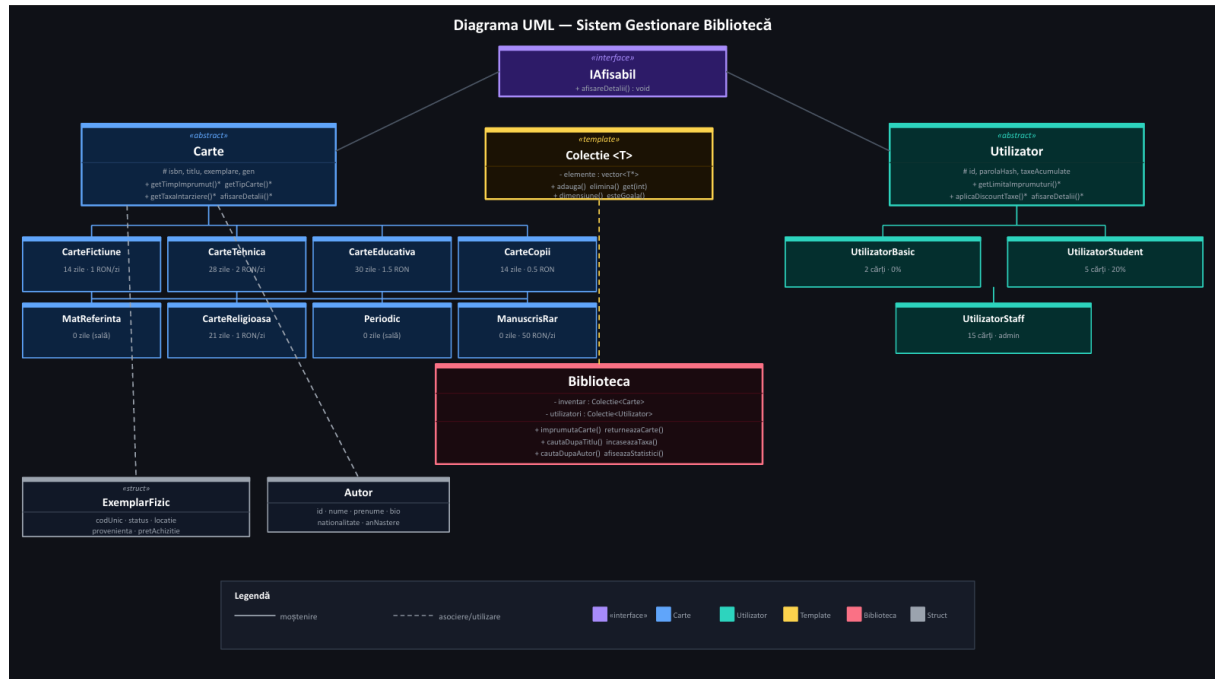
Clasă excepție	Cauză declanșare
CarteLipsaException	Carte inexistentă, toate exemplarele ocupate, sau nu se împrumută
LimitaDepastitaException	Utilizatorul a atins limita de împrumuturi simultane
IsbnDuplicatException	Se încearcă adăugarea unui ISBN deja existent
UtilizatorInexistentException	ID utilizator negăsit în sistem
TaxaInvalidaException	Sumă negativă sau zero la încasare

```
try {
    bib.imprumutaCarte("STU-1001", "ISBN-F001", 7, 4, 2024);
} catch (const LimitaDepastitaException& e) {
    cout << e.what();
} catch (const CarteLipsaException& e) {
    cout << e.what();
}
```

5. Structura Fișierelor

```
3122A_UT_GestionareBiblioteca/
src/
    IAFisabil.h          - interfata pura de afisare
    Colectie.h           - sablon generic <T>
    Autor.h              - entitate autor (unica in IndexAutori)
    Editura.h            - entitate editura (unica in IndexEdituri)
    Exceptii.h           - 5 exceptii custom
    Carte.h              - clasa abstracta de baza + ExemplarFizic
    CarteFictiune.h      - derivata: gen literar, varsta minima
    CarteTehnica.h       - derivata: domeniu, dificultate, resurse
    CarteEducativa.h     - derivata: materie, profil, clasa
    CarteCopii.h         - derivata: varsta, ilustrator, interactiv
    MaterialeReferinta.h - derivata: dictionar/atlas/enciclopedie
    TipuriSpeciale.h     - CarteReligioasa, Periodic, ManuscrisRar
    Utilizator.h         - clasa abstracta de baza + securitate
    UtilizatorBasic.h    - derivata: 2 carti, fara discount
    UtilizatorStudent.h  - derivata: 5 carti, discount 20%
    UtilizatorStaff.h    - derivata: 15 carti, roluri admin
    Biblioteca.h         - orchestratorul principal
    DataSeeder.h         - generare date de test
    Fisiere.h            - salvare/incarcare persistenta
    Server.h             - server HTTP pe port 8080
    main.cpp             - punct de intrare
tests/
    TestStoc.cpp         - 8 teste inventar fizic
    TestImprumuturi.cpp  - 9 teste logica imprumut
    TestPolimorfism.cpp  - 7 teste concepte POO
data/
    - date persistate (txt)
Makefile
README.md
```

6. Diagrama de Clase



7. Compilare și Rulare

Cerințe: g++ cu suport C++17, make, WSL Ubuntu sau Linux nativ.

Comandă	Descriere
<code>make</code>	Compilare completă
<code>make quick</code>	Compilare rapidă într-un singur pas
<code>make run</code>	Compilare și rulare imediată
<code>make clean</code>	Ștergere fișiere generate
<code>make rebuild</code>	Rebuild complet de la zero
<code>make tests</code>	Compilare și rulare toate testele
<code>make test-stoc</code>	Doar TestStoc.cpp
<code>make test-imprumuturi</code>	Doar TestImprumuturi.cpp
<code>make test-polimorfism</code>	Doar TestPolimorfism.cpp

```
g++ -std=c++17 -Wall -I./src -pthread -o biblioteca_app src/main.cpp
./biblioteca_app
# Browser: http://localhost:8080
```

La prima pornire se rulează DataSeeder-ul care generează datele de test. La pornirile ulterioare se încarcă datele salvate în `data/`.

8. Popularea cu Date (DataSeeder)

Fișier: `src/DataSeeder.h` — la pornirea aplicației, `DataSeeder::populeaza(bib)` încarcă automat peste 600 de titluri și ~200 de utilizatori. Fiecare titlu primește 1–3 exemplare fizice cu cod unic `BIB-YYYY-NNNNN` generat automat.

Tip carte	Nr. titluri	Exemple
CarteFictiune	~150	SF, Fantasy, Thriller, Clasici români și universali
CarteTehnica	~150	C++, Python, Java, AI, DevOps, Algoritmi
CarteEducativa	~175	Manuale cls. 5-12, culegeri BAC, cursuri universitare
CarteCopii	~100	Harry Potter, basme, enciclopedii copii
MaterialeReferinta	~50	Dicționare, atlase, enciclopedii
CarteReligioasa	~15	Biblie, Filocalie, Coran
Periodic	~15	Ziare și reviste curente
ManuscrisRar	~10	Cronicari moldoveni, manuscrise medievale

Utilizatori generați: 5 Staff (Bibliotecar, Bibliotecar Șef, Administrator, Casier, Arhivar), 30 Studenți (5 facultăți), 80 Basic. Toate parolele sunt stocate ca hash, nu în clar.

9. Interfața Web

Fișier: `src/Server.h` — interfață web dark academia accesibilă la `http://localhost:8080`, implementată cu `cpp-http-lib`.

Secțiune	Staff	Student / Basic
Dashboard cu statistici	Da (global)	Da (personal)
Inventar cărți cu ISBN	Da (complet + împrumut)	Da (cu împrumut direct)
Gestiune utilizatori + detalii	Da	Nu
Împrumuturi & Returnări (toți utilizatorii)	Da	Nu
Procese Verbale	Da	Nu
Statistici	Da	Nu
Profil personal + schimbare parolă	Da	Da
Istoricul meu (activ + returnat)	Da	Da
Listă de lectură (wishlist)	Da	Da
Recomandări după gen	Da	Da
Căutare avansată	Da	Da
Anunțuri	Da (adaugă)	Da (vizualizează)

10. Teste Unitare

Testele sunt organizate în 3 fișiere separate în `tests/`, fiecare compilabil independent.

TestStoc.cpp — 8 teste pentru inventarul fizic

Test	Descriere
T01	Carte fără exemplare: <code>nrExemplareTotal = 0, getPrimulDisponibil = nullptr</code>
T02	Adăugare exemplare: contorul crește corect după fiecare adăugare
T03	Format cod unic BIB-2024-NNNNN, lungime 14 caractere, coduri diferite
T04	Status exemplar devine IMPRUMUTATA la împrumut, DISPONIBILA la returnare
T05	Disponibilitate corectă cu exemplare multiple și mai mulți utilizatori
T06	MaterialeReferinta aruncă CarteLipsaException la tentativa de împrumut
T07	ISBN duplicat aruncă IsbnDuplicatException
T08	Același autor adăugat de 2 ori returnează pointer identic (unicitate index)

TestImprumuturi.cpp — 9 teste pentru logica de împrumut

Test	Descriere
T01	Împrumut simplu: utilizatorul are cartea în lista activă
T02	Limita Basic (2 cărți): al 3-lea împrumut aruncă LimitaDepastitaException
T03	Limita Student (5 cărți): al 6-lea împrumut aruncă excepție
T04	Returnarea eliberează slot: utilizatorul poate împrumuta din nou
T05	Utilizator inexistent aruncă UtilizatorInexistentException
T06	Carte fără niciun exemplar adăugat aruncă CarteLipsaException
T07	Staff: <code>aplicaDiscountTaxe()</code> returnează 0.0 RON indiferent de sumă
T08	Discount Student 20%: 10 RON brut devine 8 RON, 25 RON devine 20 RON
T09	Taxă acumulată corect după returnare cu întârziere, zero după plată

TestPolimorfism.cpp — 7 teste pentru concepte POO

Test	Descriere
T01	<code>getTipCarte()</code> returnează tipul corect pentru toate cele 8 derivate
T02	<code>getTimpImprumut()</code> diferit per tip prin pointer de bază <code>Carte*</code>
T03	<code>getLimitaImprumuturi()</code> diferit per tip prin pointer <code>Utilizator*</code>
T04	<code>areDreptDeAdmin()</code> false pentru Basic/Student, true pentru Staff
T05	<code>Colectie<T></code> : dimensiune, get, out_of_range, invalid_argument la null
T06	<code>IAfisabil*</code> : <code>afisareDetalii()</code> funcționează prin pointer de interfață
T07	Operator <code><<</code> supraîncărcată produce output nevid cu date corecte

```
make tests          # toate 3 fisierele
make test-stoc      # doar TestStoc
make test-imprumuturi # doar TestImprumuturi
make test-polimorfism # doar TestPolimorfism
```

11. Cerințe Acoperite

Cerințe obligatorii:

	Cerință	Detaliu implementare
✓	Clase Carte, Utilizator, Biblioteca	Implementate complet cu atribute private și metode publice
✓	Moștenire	8 tipuri de cărți și 3 tipuri de utilizatori din clase abstracte
✓	Polimorfism	<code>afisareDetalii()</code> , <code>getTipCarte()</code> , <code>getLimitaImprumuturi()</code> , <code>aplicaDiscountTaxe()</code>
✓	Encapsulare	Toate atributele private/protected, parola hash niciodată expusă direct
✓	Evenimente logate	Împrumut/returnare/adăugare/încasare scrise în <code>biblioteca_log.txt</code>
✓	Git — 5+ commit-uri	Branch develop, commit-uri descriptive pe GitHub

Cerințe facultative:

	Cerință	Detaliu implementare
✓	Șablon generic	<code>Colectie<T></code> înlocuiește <code>vector<T*></code> brut, cu gestionare memorie automată
✓	Excepții custom	5 clase de excepții derivate din <code>std::runtime_error</code> în <code>Exceptii.h</code>

12. Decizii de Proiectare

De ce Autor și Editura ca entități, nu ca string?

Un string Frank Herbert apare în zeci de cărți. Dacă vrem să actualizăm bio-ul sau să adăugăm un link, cu string ar trebui modificat în fiecare carte. Cu entitate separată, modificăm o singură dată și toate cărțile reflectă automat schimbarea.

De ce ExemplarFizic separat de Carte?

ISBN-ul este același pentru toate copiile unui titlu, dar fiecare copie fizică are locație proprie, stare proprie și cod propriu. Fără ExemplarFizic, nu putem ști că exemplarul de pe raftul A-3 este împrumutat dar cel de pe A-7 este disponibil.

De ce IAfisabil ca interfață separată?

Carte și Utilizator nu au o relație de moștenire între ele, dar ambele trebuie să se poată afișa. IAfisabil permite scrierea de funcții generice care acceptă orice IAfisabil* fără să știe dacă e carte sau utilizator.

De ce Colectie<T> în loc de vector<T*>?

vector<T*> nu gestionează memoria — dacă uiți delete, ai memory leak. Colectie<T> face delete automat în destructor, interzice copierea accidentală care ar face double-delete, și validează null-ul la adăugare.

De ce parola ca hash și nu în clar?

Dacă fișierul de date sau memoria este citită de un atacator, parolele în clar sunt compromise instant. Hash-ul nu permite recuperarea parolei originale — verificarea se face comparând hash-uri, nu parole.

De ce istoricul este sortat automat la fiecare adaugaEveniment()?

Evenimentele pot fi adăugate în orice ordine. Fără sortare automată, istoricul ar apărea în ordinea inserției, nu cronologic. Sortând la fiecare inserare garantăm că vectorul este mereu ordonat indiferent de cine și când adaugă evenimente.